

גרף תוכנית (Program Graph)

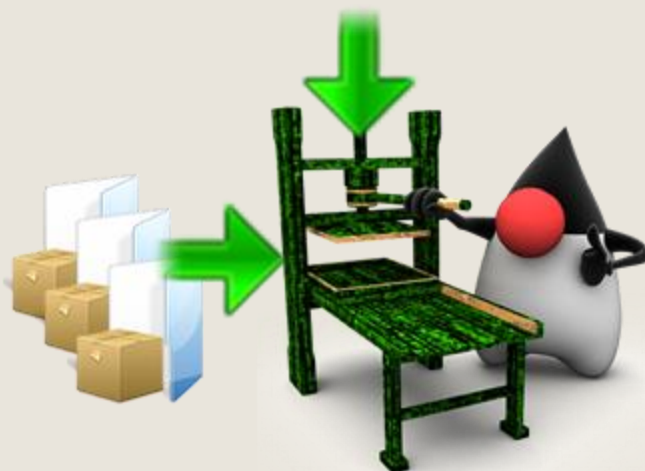
גרא וייס
המחלקה למדעי המחשב
אוניברסיטת בן-גוריון





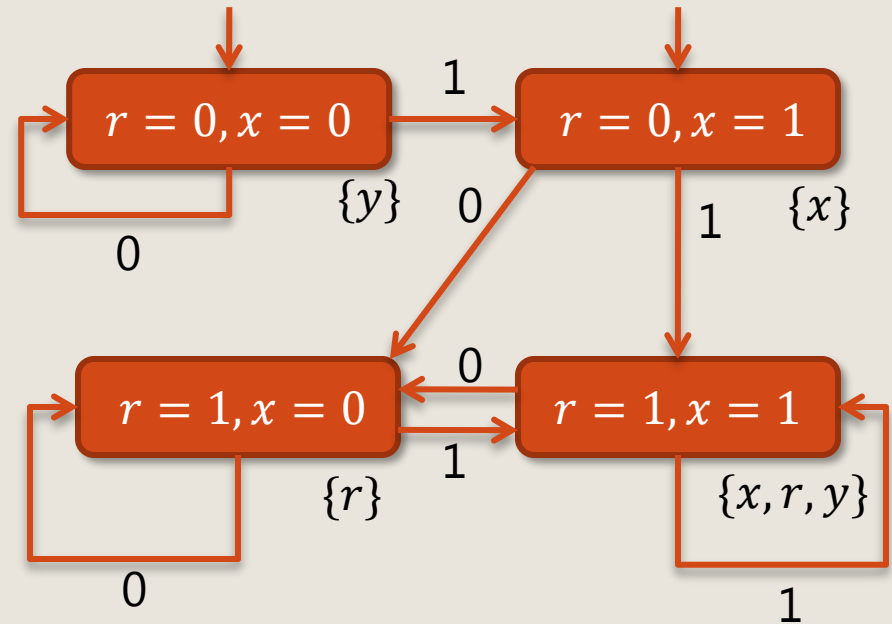
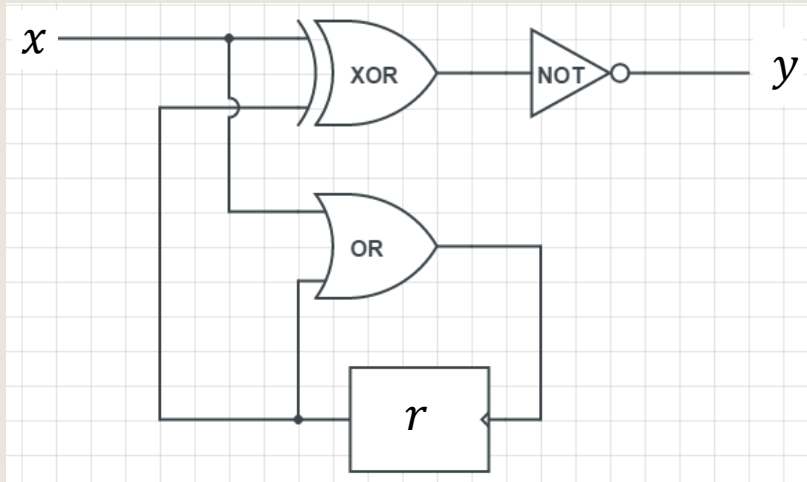
בהרבה מקרים, לא נרצה לכתוב
מערכות מעברים באופן ישיר

נעדיף לתאר את המערכת בשפה "עילית"
ולהשתמש בקומפיילר שייצר מערכת מעברים





דוגמה: תרגום מסכמת מעגל לוגי למערכת מעברים

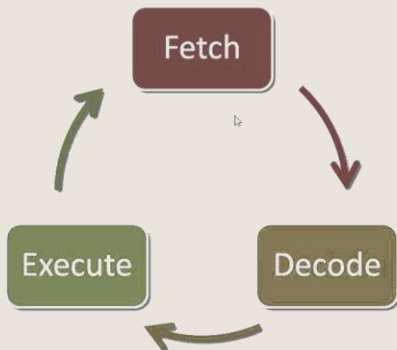


בכל צעד (תְּקִיּוֹק שְׁעוֹן):

1. מעדכנים את ערכי האוגרים לפי הקלט הקודם

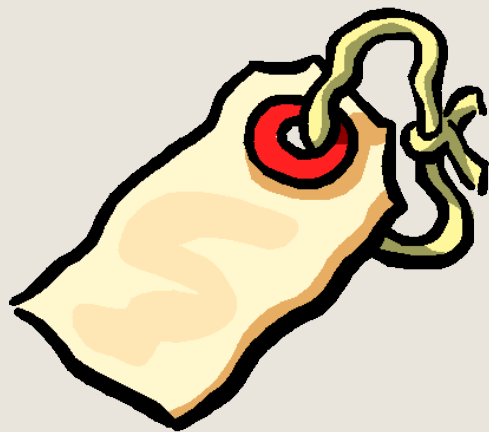
2. קובעים את הפלט

3. קוראים קלט חדש





תיוג המצבים לפסוקים אטומיים



שתי אפשרויות לתיוג המצבים:

• נבחר $AP = \{x, y, r\}$

$$L(\langle x = 1, r = 1 \rangle) = \{x, r, y\}$$

$$L(\langle x = 0, r = 1 \rangle) = \{r\}$$

$$L(\langle x = 1, r = 0 \rangle) = \{x\}$$

$$L(\langle x = 0, r = 0 \rangle) = \{y\}$$

• תכונה לדוגמה: "ברגע שהאוגר הוא אחד הוא נשאר אחד"

• נבחר $AP = \{x, y\}$ – עכשיו ערכי האוגר "בלתי נראים"

$$L(\langle x = 1, r = 1 \rangle) = \{x, y\}$$

$$L(\langle x = 0, r = 1 \rangle) = \emptyset$$

$$L(\langle x = 1, r = 0 \rangle) = \{x\}$$

$$L(\langle x = 0, r = 0 \rangle) = \{y\}$$

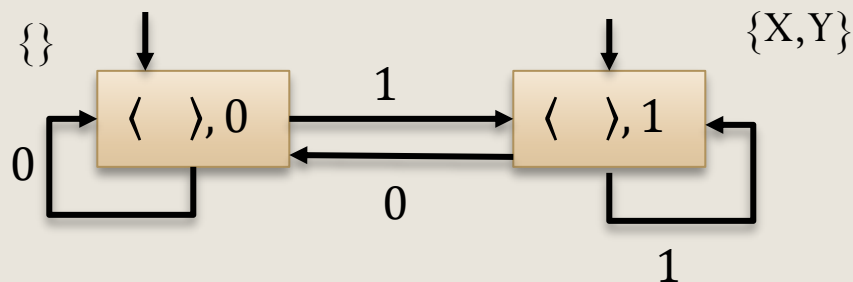
• תכונה לדוגמה: "ערך משתנה הפלט y מתחלף אינסוף פעמים"

• אי אפשר לתאר תכונות המתייחסות לערך האוגר



דוגמה נוספת: חוט בודד

X ————— Y

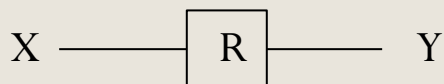


$upd(<>, x) = trivial$

$out(<>, x) = \langle x \rangle$

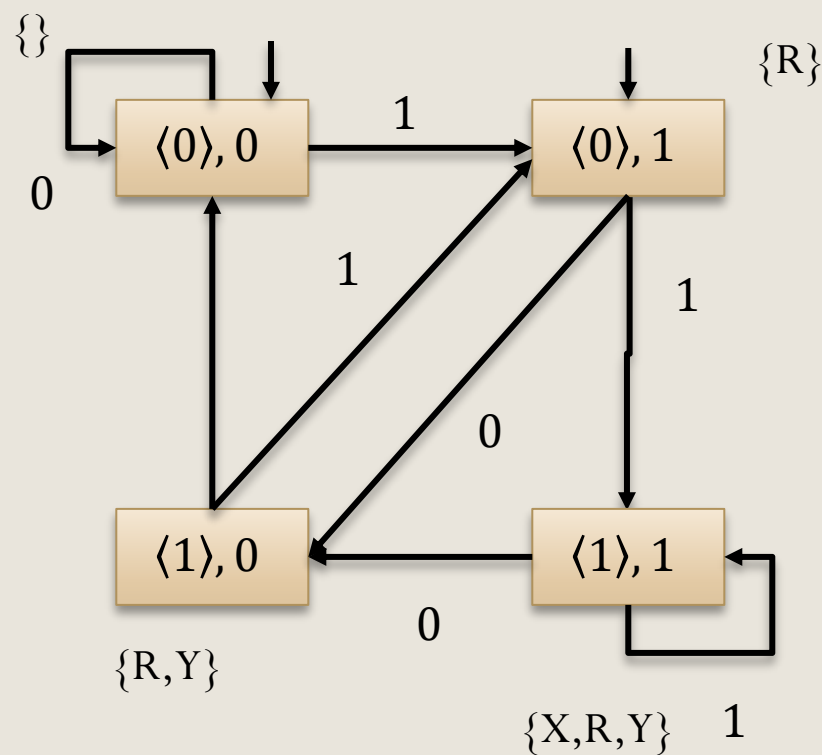


דוגמה נוספת: מעגל מעכב



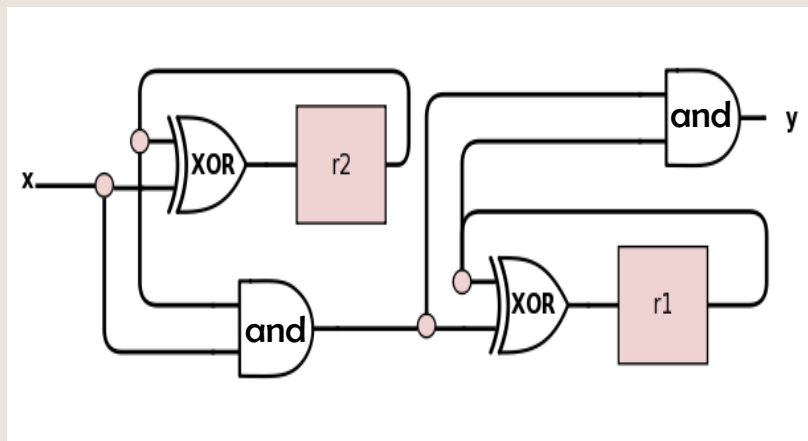
$$upd(\langle R \rangle, x) = \langle x \rangle$$

$$out(\langle R \rangle, x) = \langle R \rangle$$





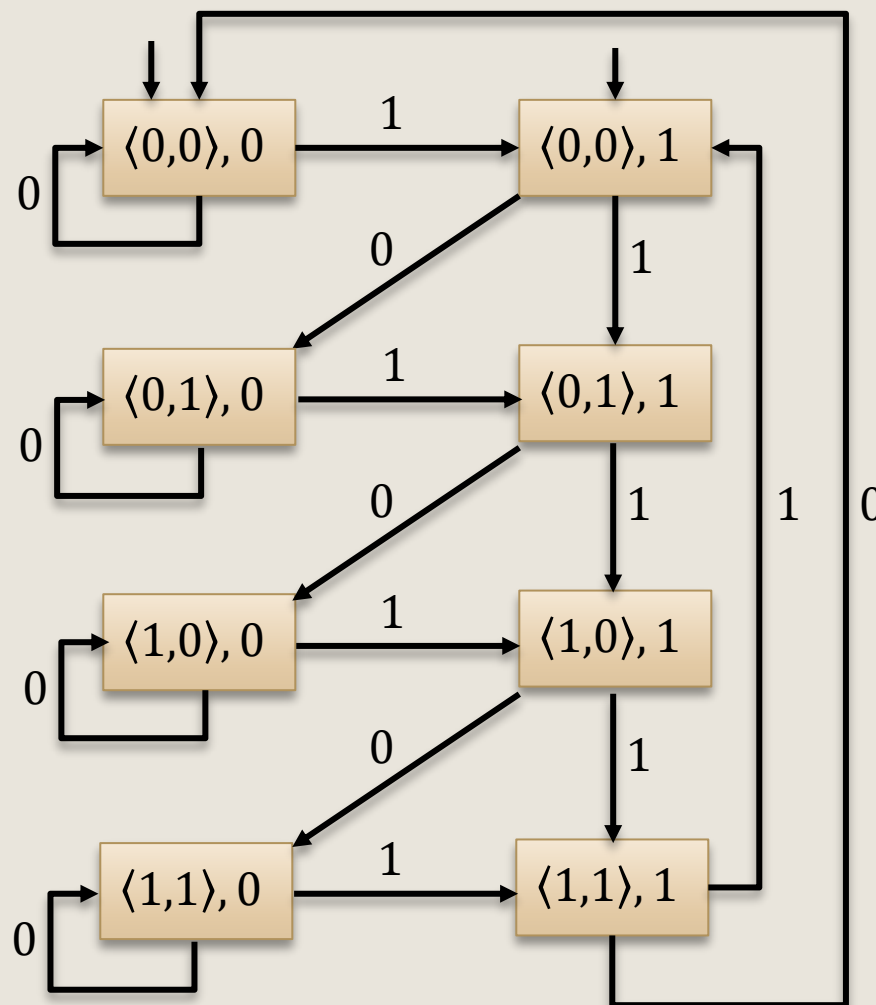
דוגמה נוספת: מעגל מונה



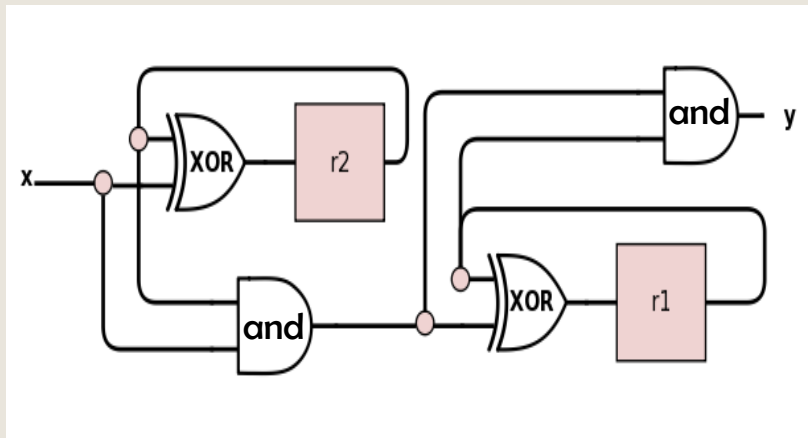
$$upd(\langle r_1, r_2 \rangle, x) = \langle (x \wedge r_2) \oplus r_1, x \oplus r_2 \rangle$$

$$out(\langle r_1, r_2 \rangle, x) = \langle r_1 \wedge r_2 \wedge x \rangle$$

איך נְתִיג את המצבים?



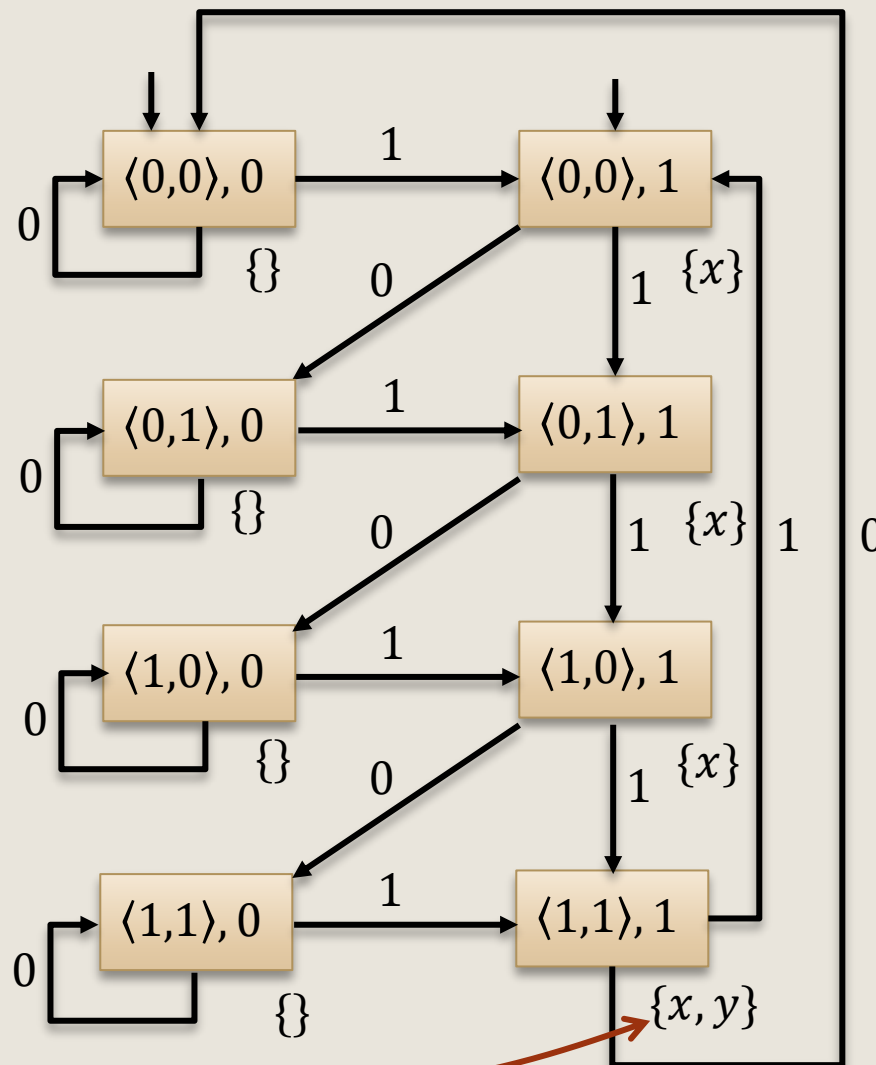
דוגמה נוספת: מעגל מונה



$$upd(\langle r_1, r_2 \rangle, x) = \langle (x \wedge r_2) \oplus r_1, x \oplus r_2 \rangle$$

$$out(\langle r_1, r_2 \rangle, x) = \langle r_1 \wedge r_2 \wedge x \rangle$$

המונה נותן אות בכל פעם
נשפך ארבעה מופעים של 1





מתכון לתרגום סכימת מעגל לוגי למערכת מעברים

- הרחבת הרעיון שהוצג בדוגמה בשקפים הקודמים



- ניתן לממש מהדר (compiler):

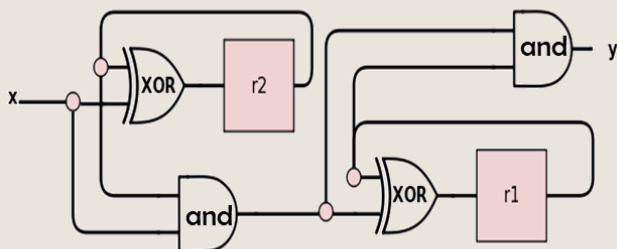
- קלט: סכימת מעגל לוגי

- פלט: מערכת מעברים

- דוגמה ל-"שפה עילית" המותאמת לתחום הבעיה
(problem domain)

- מאפשר שימוש בכלים שנפתח בהקשר רחב:
בהנחה שהתרגום מעולם המושגים של הבעיה נכון

דוגמה: מידול מעגלים לוגיים



#reg = 2
#in = 1
#out = 1

$$\text{upd}(\langle r_1, r_2 \rangle, x) = \langle (x \wedge r_2) \oplus r_1, x \oplus r_2 \rangle$$

$$\text{out}(\langle r_1, r_2 \rangle, x) = \langle r_1 \wedge r_2 \wedge x \rangle$$

$$S = \{0,1\}^{\#reg} \times \{0,1\}^{\#in}$$

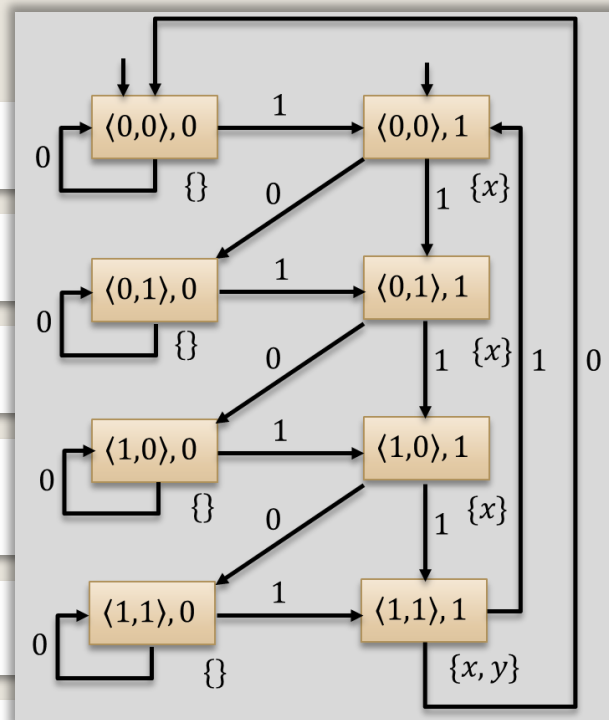
$$Act = \{0,1\}^{\#in}$$

$$I = \{0\}^{\#reg} \times \{0,1\}^{\#in}$$

$$\rightarrow = \{ \langle r, x \rangle \xrightarrow{x'} \langle \text{upd}(r, x), x' \rangle : \langle r, x \rangle \in S \}$$

$$AP = \{ r_1, \dots, r_{\#reg}, x_1, \dots, x_{\#in}, y_1, \dots, y_{\#out} \}$$

$$L(s = \langle \langle r_1, \dots, r_{\#reg} \rangle, \langle x_1, \dots, x_{\#in} \rangle \rangle) = \{ r_i : r_i = 1 \} \cup \{ x_i : x_i = 1 \} \cup \{ y_i : y_i = 1, \langle y_1, \dots, y_{\#out} \rangle = \text{out}(s) \}$$



תרגום סכימת מעגל לוגי למערכת מעברים



```
public interface Circuit {
```

```
    ...  
    List<Boolean> updateRegisters(List<Boolean> registers, List<Boolean> inputs);  
    List<Boolean> computeOutputs(List<Boolean> registers, List<Boolean> inputs);
```

```
}
```

```
public class ExampleCircuit implements Circuit {
```

```
    ...  
    public List<Boolean> updateRegisters(List<Boolean> registers, List<Boolean> inputs) {  
        List<Boolean> r = new LinkedList<>()  
        r.add(registers.get(0) || inputs.get(0));  
        return r;  
    }
```

```
    public List<Boolean> computeOutputs(List<Boolean> registers, List<Boolean> inputs)  
        List<Boolean> y = new LinkedList<>();  
        y.add(!(registers.get(0) ^ inputs.get(0)));  
        return y;  
    }
```

```
}
```

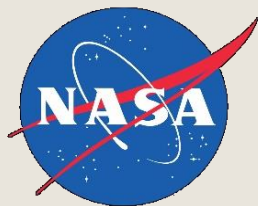
```
TransitionSystem ts = fvmFacade.transitionSystemFromCircuit(c);
```



למרות שהשימוש במערכות מעברים ובטכניקת
בדיקות-מודל הוכיח עצמו ונמצא בשימוש תעשייתי
בעיקר בתחום מעגלי החומרה



אנחנו נתרכז בקורס בשימושים בתחום התוכנה



Microsoft

לכן, נגדיר תרגום ממבנה שנקרא "גרף תוכנית"
(סוג של תרשים זרימה) למערכת מעברים



מעברים מִתְנִים במערכת מעברים

- תוכניות תלויות-נתונים בנויות מפקודות מהצורה:

```
if  $x \% 2 = 1$  then  $x := 3x + 1$  else  $x := x/2$  fi
```

ראו: https://he.wikipedia.org/wiki/השערת_קולץ

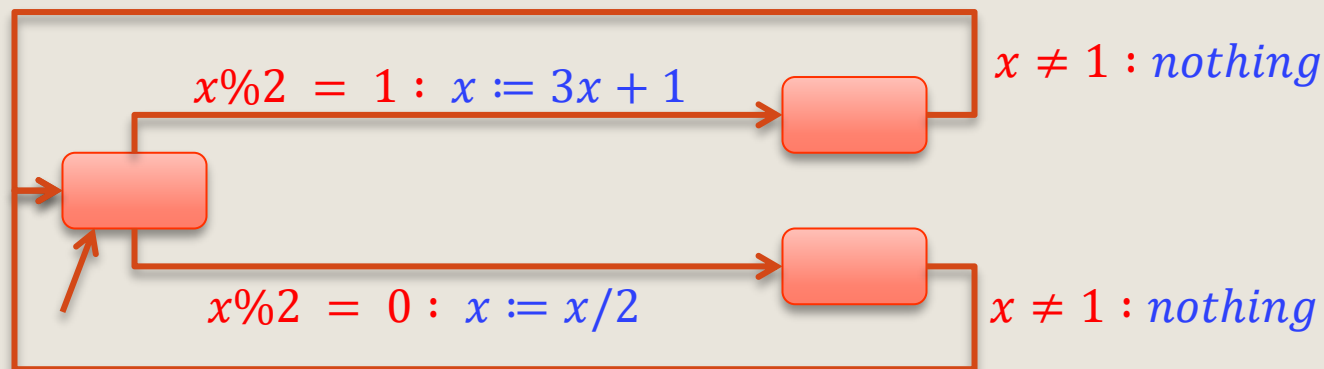
- אפשרויות למידול כמערכת מעברים:

- אפשר **להשמיט** את התנאים ולתרגם למעבר לא דטרמיניסטי

- אפשר להגדיר **שפה עילית נוספת** שתתורגם למערכת מעברים:



זאת לא
מערכת מעברים!



- נקרא לזה **גרף תוכנית (program graph)**

- בהמשך נראה איך פורסים (unfold) גרף-תוכנית למערכת מצבים (באופן אוטומטי)

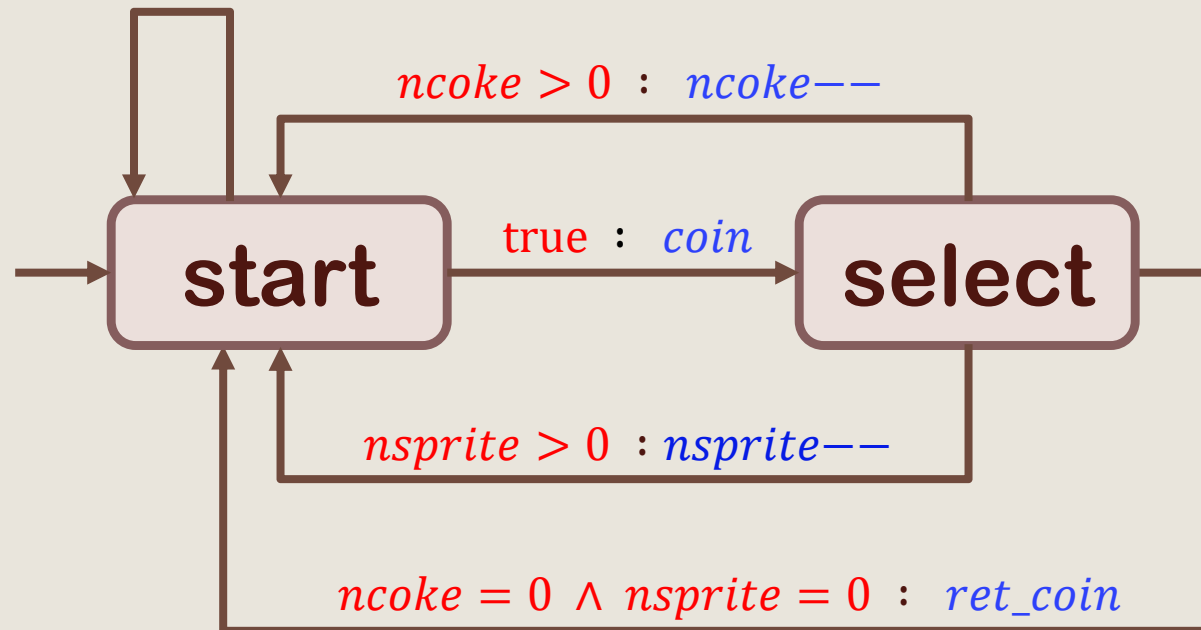
דוגמה: שכלול מודל מכונת המשקאות

(אנחנו בונים כאן **גרף תוכנית**, לא מערכת מעברים)



סופרים כמות בקבוקי הספרייט והקולה ומחזירים מטבע אם המכונה ריקה:

true : $ncoke = 10 \wedge nsprite = 10$



דוגמה: שכלול מודל מכונת המשקאות

(אנחנו בונים כאן **גרף תוכנית**, לא מערכת מעברים)



סופרים כמות בקבוקי הספרייט והקולה ומחזירים מטבע אם המכונה ריקה:

$start \xrightarrow{\text{true} : \text{coin}} select$

$start \xrightarrow{\text{true} : \text{refill}} start$

$select \xrightarrow{\text{nsprite} > 0 : \text{get_sprite}} start$

$select \xrightarrow{\text{ncoke} > 0 : \text{get_coke}} start$

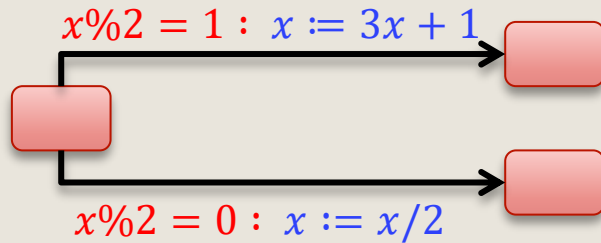
$select \xrightarrow{\text{ncoke} = 0 \wedge \text{nsprite} = 0 : \text{ret_coin}} start$



פעולה	השפעה על משתנים
<i>coin</i>	
<i>ret_coin</i>	
<i>get_sprite</i>	$\text{nsprite} := \text{nsprite} - 1$
<i>get_coke</i>	$\text{ncoke} := \text{ncoke} - 1$
<i>refill</i>	$\text{nsprite} := \text{max}; \text{ncoke} := \text{max}$



גרף תוכנית



- כלי למידול תוכניות עם משתנים ותנאים

- גרף מִכָּוֵן עם סימונים על הקשתות:

- הצמתים הם המקומות בתוכנית = ערך של ה-program counter

- הקשתות מסומנות בביטויים מהצורה פעולה : תנאי

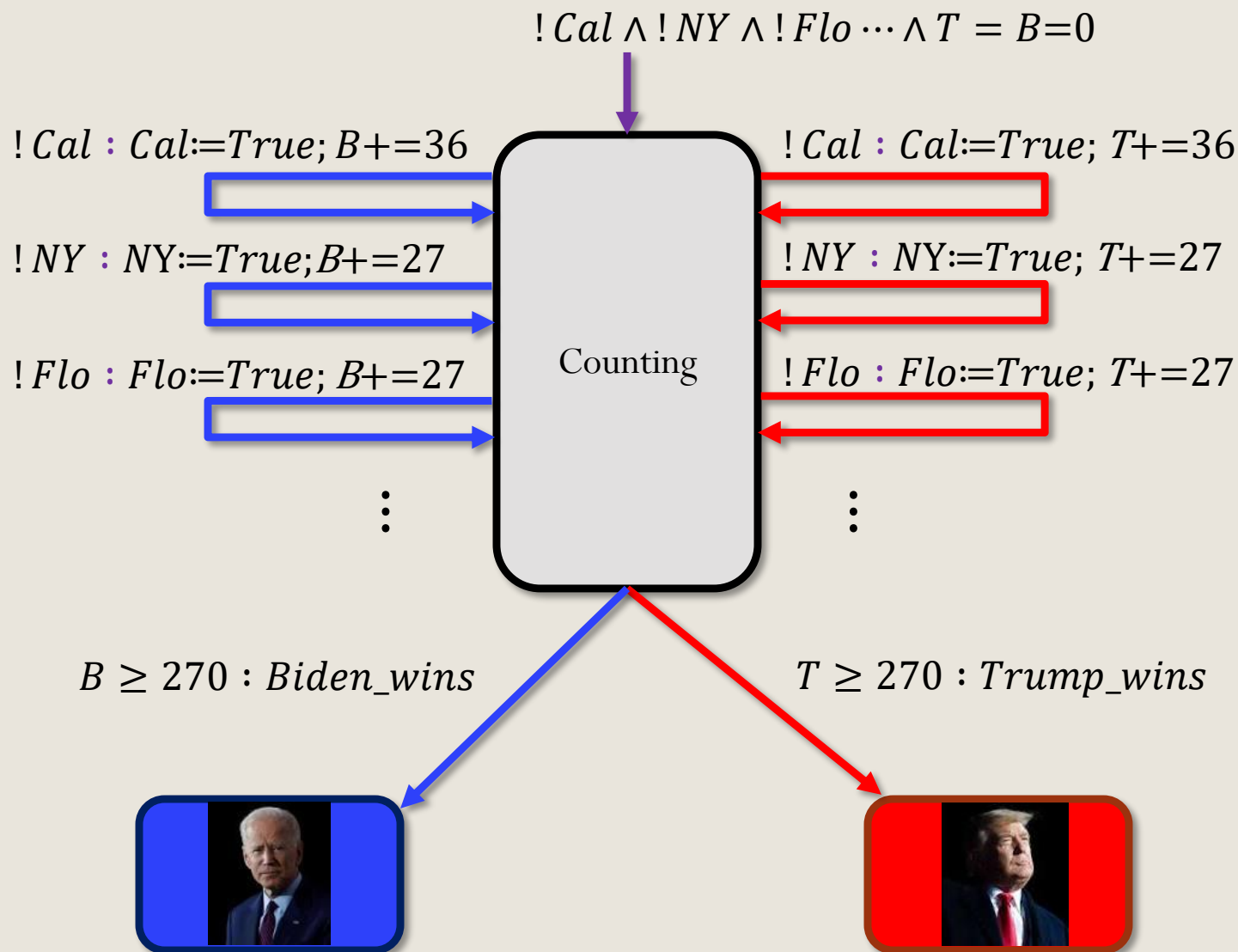
- התנאים קובעים איזה קשתות "מאפשרות"

- הפעולות מעדכנות את ערכי המשתנים

אנחנו נתרגם גרפי תוכנית למערכות מעברים שיינתנו כקלט ל-Model Checker



דוגמה: ספירת הקולות בארה"ב





$|Reach(TS)|$ חישוב

```

reps = [1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 4, 4, 4, 4, 4, 4, 5, 5, 5, 6,
        6, 7, 7, 7, 8, 8, 8, 8, 9, 9, 9, 9, 10, 11, 12, 13, 14, 14, 16, 18, 18, 27, 27, 36]

I = {(0, 0, tuple((False for i in range(49))))}

def markTrue(flags, i):
    return flags[:i] + (True,) + flags[i + 1:]

def post(s):
    return (set() if s[0] >= 270 or s[1] >= 270 else
            ((s[0]+reps[i], s[1], markTrue(s[2], i)) for i in range(49) if not s[2][i]) |
            {(s[0], s[1]+reps[i], markTrue(s[2], i)) for i in range(49) if not s[2][i]})

def POST(S):
    return set.union(*[post(s) for s in S])

S = I
for i in range(3):
    print(f'{len(S)=}')
    S = POST(S)

```

$\text{len}(S)=1$
 $\text{len}(S)=98$
 $\text{len}(S)=???$

1. $98 \cdot 48 \cdot 2$
2. $4 \cdot \binom{49}{2} - 99$
3. ...



$|Reach(TS)|$ חישוב

```

reps = [1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 4, 4, 4, 4, 4, 4, 5, 5, 5, 6,
        6, 7, 7, 7, 8, 8, 8, 8, 9, 9, 9, 9, 10, 11, 12, 13, 14, 14, 16, 18, 18, 27, 27, 36]

I = {(0, 0, tuple((False for i in range(49))))}

def markTrue(flags, i):
    return flags[:i] + (True,) + flags[i + 1:]

def post(s):
    return (set() if s[0] >= 270 or s[1] >= 270 else
            ((s[0]+reps[i], s[1], markTrue(s[2], i)) for i in range(49) if not s[2][i]) |
            {(s[0], s[1]+reps[i], markTrue(s[2], i)) for i in range(49) if not s[2][i]})

def POST(S):
    return set.union(*[post(s) for s in S])

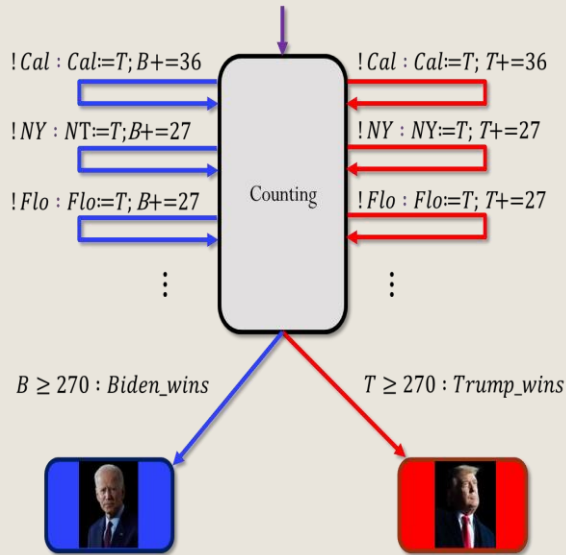
S = I
for i in range(3):
    print(f'{len(S)=}')
    S = POST(S)

```

$len(S)=1$
 $len(S)=98$
 $len(S)=4633$
 $len(S)=139085$



הגדרות פורמליות: הצבות, תנאים והשפעה



• נקרא לקבוצת המשתנים שבתוכנית Var

• לקבוצת ההשמות (evaluations): $Eval(Var)$

• השמה היא פונקציה המתאימה לכל משתנה ערך.

• נתאר בד"כ כקבוצת זוגות: $\{x \mapsto 3, y \mapsto 'Hi'\}$

• לקבוצת התנאים (conditions): $Cond(Var)$

• $\models: Cond(Var) \times Eval(Var) \rightarrow \{True, False\}$

• פונקציה הקובעת ערך אמת לכל תנאי ולכל השמה

• $\models (x < 9, \{x \mapsto 3, y \mapsto 'Hi'\}) = True$

• $\{x \mapsto 3, y \mapsto 'Hi'\} \models x < 9$

• $\{x \mapsto 3, y \mapsto 'Hi'\} \not\models x < 2$

• לקבוצת הפעולות: Act

• $Effect: Act \times Eval(Var) \rightarrow Eval(Var)$

• פונקציה המתאימה לכל פעולה ולכל השמה את ההשמה שתהיה לאחר הפעולה.

• $Effect(x++, \{x \mapsto 3, y \mapsto 'Hi'\}) = \{x \mapsto 4, y \mapsto 'Hi'\}$

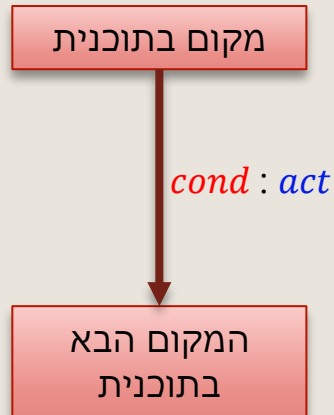


הגדרה: גרף תוכנית

גרף תוכנית (program graph) מעל קבוצת משתנים Var הוא שביעיה סדורה

$$\langle Loc, Act, \models, Effect, \rightarrow, Loc_0, g_0 \rangle$$

באשר:



• Loc היא קבוצת המקומות (locations)

• $Loc_0 \subseteq Loc$ היא קבוצת מקומות ההתחלה

• Act היא קבוצת הפעולות

• $\models: Cond(Var) \times Eval(Var) \rightarrow \{True, False\}$ היא פונקציית הערכת התנאים

• $Effect: Act \times Eval(Var) \rightarrow Eval(Var)$ היא פונקציית ההשפעה

• $\rightarrow \subseteq Loc \times Cond(Var) \times Act \times Loc$ הוא יחס המעברים

• $g_0: Loc_0 \rightarrow Cond(Var)$ הוא תנאי ההתחלה

גרף תוכנית למכונת משקאות המשוכללת



$Var = \{nsprite, ncoke\}$

$Loc = \{start, select\}$

$Act = \{get_coke, get_sprite, coin, ret_coin, refill\}$

$Effect(coin, \eta) = \eta$

$Effect(ret_coin, \eta) = \eta$

$Effect(refill, \eta) = [nsprite := 100, ncoke := 100]$

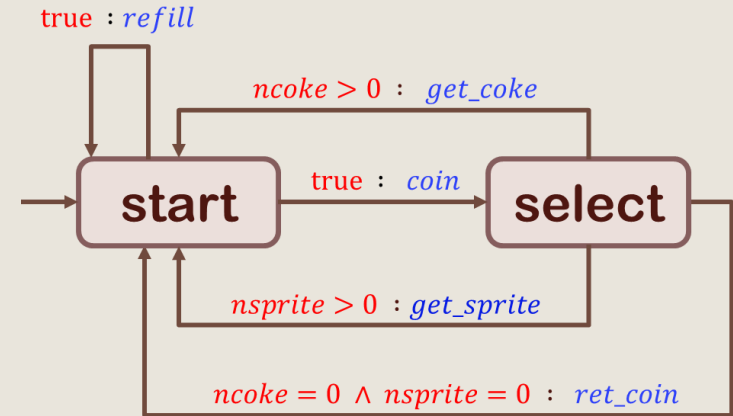
$Effect(get_sprite, \eta) = \eta[nsprite := nsprite - 1]$

$Effect(get_coke, \eta) = \eta[ncoke := ncoke - 1]$

$Loc_0 = \{start\}$

$g_0 = (nsprite = 100 \wedge ncoke = 100)$

סימון לבניית פונקציה חדשה הנבדלת
מאחרת רק בערך מסוים



יחס המעברים:

$start \xrightarrow{\text{true : coin}} select$

$start \xrightarrow{\text{true : refill}} start$

$select \xrightarrow{ncoke > 0 : get_coke} start$

$select \xrightarrow{nsprite > 0 : get_sprite} start$

$select \xrightarrow{ncoke = 0 \wedge nsprite = 0 : ret_coin} start$



```
VENDING_PG = {
  'Loc': {'start', 'select'},
  'Loc0': {'start'},
  'Act': {'coin', 'refill', 'get_coke', 'get_sprite', 'ret_coin'},
  'Eval': evaluate,
  'Effect': effect,
  'to': {
    ('start', 'true', 'coin', 'select'),
    ('start', 'true', 'refill', 'start'),
    ('select', 'ncoke > 0', 'get_coke', 'start'),
    ('select', 'nsprite > 0', 'get_sprite', 'start'),
    ('select', 'ncoke=0 && nsprite=0', 'ret_coin', 'start')
  },
  'g0': {'ncoke':10, 'nsprite':10}
}
```

קוד לבניית גרף התוכנית

פונקציית פירוש תנאים ($\models$)

```
def evaluate( cond, eta ) :
  return {
    'true': lambda eta: True,
    'ncoke > 0': lambda eta: eta['ncoke']>0,
    'nsprite > 0': lambda eta: eta['nsprite']>0,
    'ncoke=0 && nsprite=0': lambda eta: eta['ncoke']==0 and eta['nsprite']==0
  }[cond](eta)
```

```
def effect( act, eta ) :
  return {
    'coin': lambda eta: eta,
    'ret_coin': lambda eta: eta,
    'refill': lambda eta: {'ncoke':10, 'nsprite':10},
    'get_coke': lambda eta: eta | {'ncoke': eta['ncoke']-1},
    'get_sprite': lambda eta: eta | {'nsprite': eta['nsprite']-1},
  }[act](eta)
```

פונקציית פירוש פעולות



קוד לבניית גרף התוכנית

```
pg.addLocation(start);
pg.addLocation(select);

pg.addInitialLocation(start);

pg.addTransition(new PGTransition(start, "true", "coin", select));
pg.addTransition(new PGTransition(start, "true", "refill", start));
pg.addTransition(new PGTransition(select, "nsoda > 0", "sget", start));
pg.addTransition(new PGTransition(select, "nbeer > 0", "bget", start));
pg.addTransition(new PGTransition(select, "nbeer=0 && nsoda=0", "ret_coin", start));
```

```
public boolean evaluate(Map<String, Object> eval, String condition) {
    if (condition.equals("nbeer=0 && nsoda=0"))
        return ((int) eval.get("nsoda") == 0) && ((int) eval.get("nbeer") == 0);
    ...
}
```

פונקציית פירוש תנאים

```
public Map<String, Object> effect(Map<String, Object> eval, String action) {
    if (action.equals("refill"))
        return new HashMap<String, Object>(eval) {{
            put("nsoda", max);
            put("nbeer", max);
        }};
    ...
}
```

פונקציית פירוש פעולות



תרגום מגרף תוכנית למערכת מעברים

תיאור מילולי של הבניה

• מצבים: פריסה (unfolding)

- מצב הוא זוג סדור $\langle l, \eta \rangle$ באשר l הוא מקום בגרף התוכנית ו η היא השמה למשתנים
- מצב התחלתי הוא $\langle l_0, \eta_0 \rangle$ באשר l_0 מקום התחלתי ו η_0 מקיימת את תנאי ההתחלה g_0

• פסוקים ותיוג במערכת המעברים שאנחנו מייצרים:

• פסוקים: מקומות בגרף התוכנית ותנאים מתוך $Cond(Var)$

• $\langle l, \eta \rangle$ מתוייג ב- l ובכל התנאים המתקיימים ב η

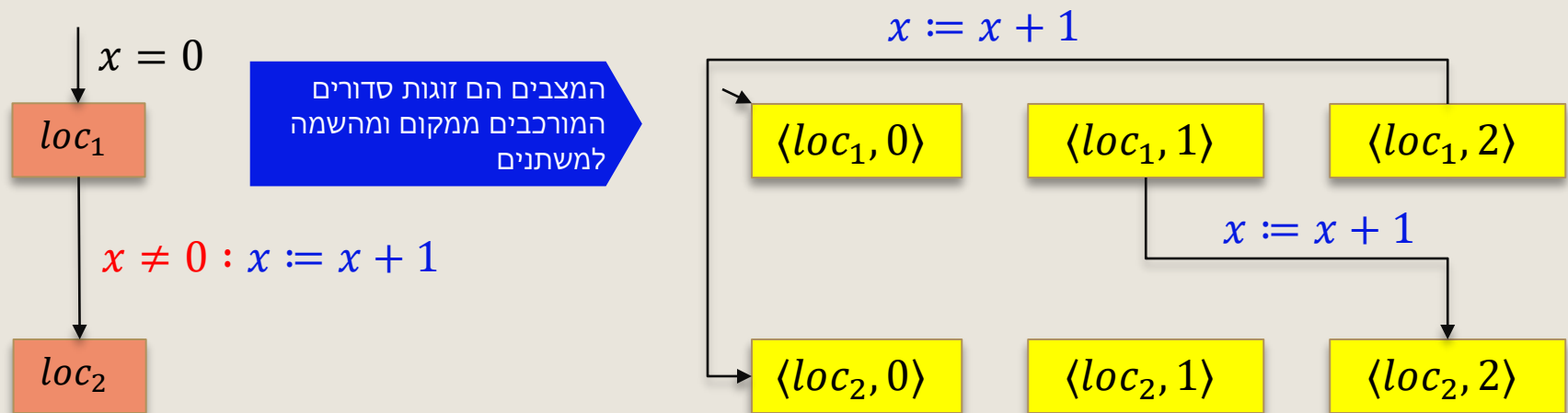


גרף תוכנית ← מערכת מעברים

• מעברים במערכת המעברים שאנחנו מייצרים:

• אם $l \xrightarrow{g:\alpha} l'$ בגרף התוכנית ו g מתקיים ב η אז מוסיפים את המעבר $\langle l, \eta \rangle \xrightarrow{\alpha} \langle l', Effect(\alpha, \eta) \rangle$

תרגום אוטומטי מגרף תוכנית למערכת מעברים



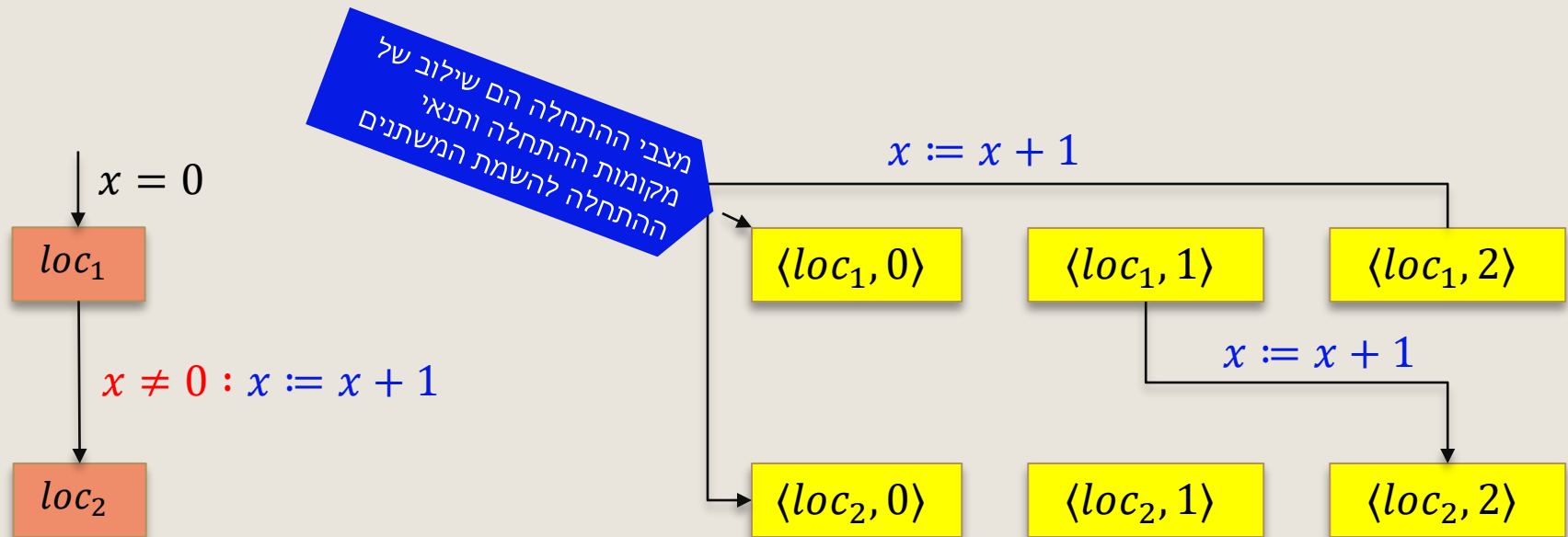
PG

$TS(PG)$

נקרא לפעולה זאת פריסה



תרגום אוטומטי מגרף תוכנית למערכת מעברים



PG

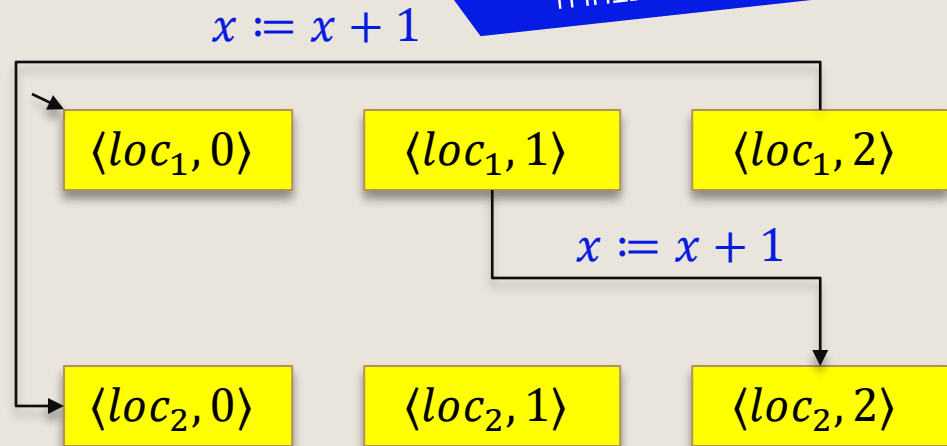
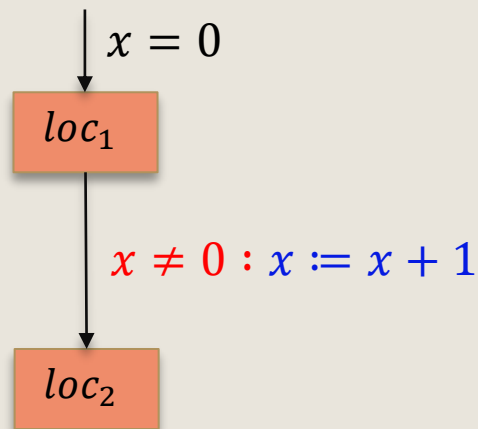
$TS(PG)$

נקרא לפעולה זאת פריסה

תרגום אוטומטי מגרף תוכנית למערכת מעברים



מעבר מחליף מקום ומשנה את
ההשמה לפי הפעולה שנבחרה



PG

$TS(PG)$

נקרא לפעולה זאת פריסה

סמנטיקת פעולה פורמאלית

כללי היסק



משמעות הסימון : $\frac{\text{הנחה}}{\text{מסקנה}}$

- אם הפסוק מעל הקו (ההנחה) מתקיים, אז הפסוק מתחת לקו (המסקנה) נכון
- פסוקי "בהנחה ש ..., אז ..." כאלה נקראים כללי היסק (Inference Rules)
- כשההנחה היא טאוטולוגיה, אפשר להשמיט אותה (וגם את הקו המפריד)
- המקרה האחרון נקרא אקסיומה



מגרף תוכנית למערכת מעברים

הגדרה פורמאלית של פעולת הפריסה

בהנתן גרף תוכנית PG , נגדיר מערכת מעברים $TS(PG)$:

• מצבים: $S = Loc \times Eval(Var)$

• יחס מעברים: $S \times Act \times S \rightarrow \subseteq$ מוגדר ע"י

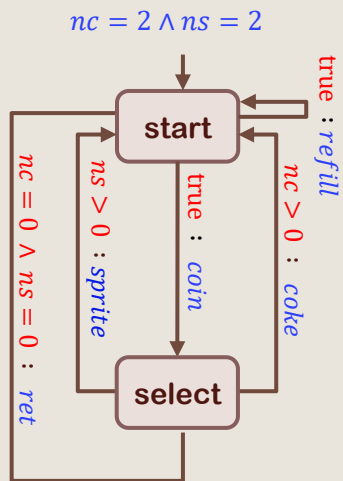
$$\frac{l \xrightarrow{g:\alpha} l' \wedge \eta \models g}{\langle l, \eta \rangle \xrightarrow{\alpha} \langle l', Effect(\alpha, \eta) \rangle}$$

• מצבי התחלה: $I = \{\langle l, \eta \rangle : l \in Loc_0, \eta \models g_0(l)\}$

• פסוקים אטומיים: $AP = Loc \cup Cond(Var)$

• תיוג: $L(\langle l, \eta \rangle) = \{l\} \cup \{g \in Cond(Var) : \eta \models g\}$

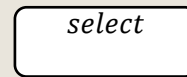
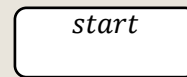
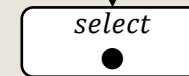
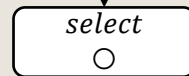
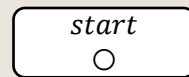
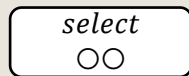
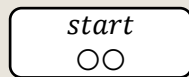
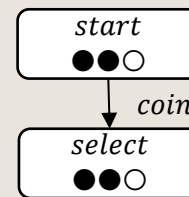
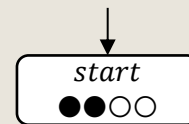
דוגמה: פריסת מערכת המעברים למכונת המשקאות



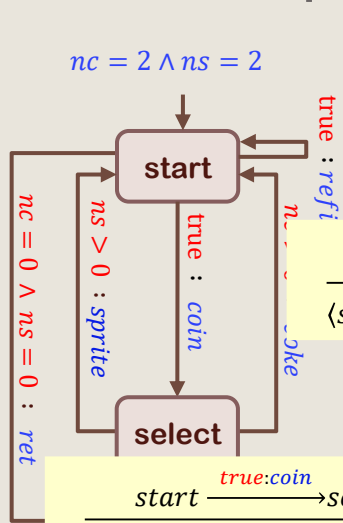
PG

$$\frac{l \xrightarrow{g:\alpha} l' \quad \wedge \quad \eta \models g}{\langle l, \eta \rangle \xrightarrow{\alpha} \langle l', \text{Effect}(\alpha, \eta) \rangle}$$

$TS(PG)$

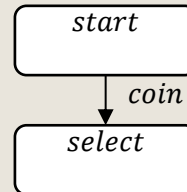
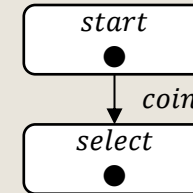
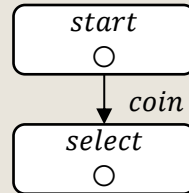
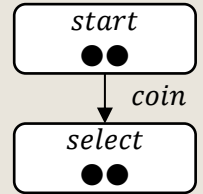
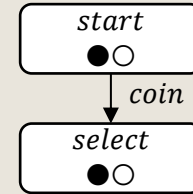
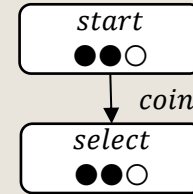
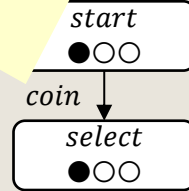
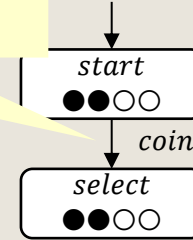


דוגמה: פריסת מערכת המעברים למכונת המשקאות



$$\frac{start \xrightarrow{true:coin} select \quad \wedge \quad \{nc \mapsto 2, ns \mapsto 2\} \models true}{\langle start, \{nc \mapsto 2, ns \mapsto 2\} \rangle \xrightarrow{coin} \langle select, \{nc \mapsto 2, ns \mapsto 2\} \rangle}$$

$$\frac{start \xrightarrow{true:coin} select \quad \wedge \quad \{nc \mapsto 1, ns \mapsto 2\} \models true}{\langle start, \{nc \mapsto 1, ns \mapsto 2\} \rangle \xrightarrow{coin} \langle select, \{nc \mapsto 1, nsprite \mapsto 2\} \rangle}$$

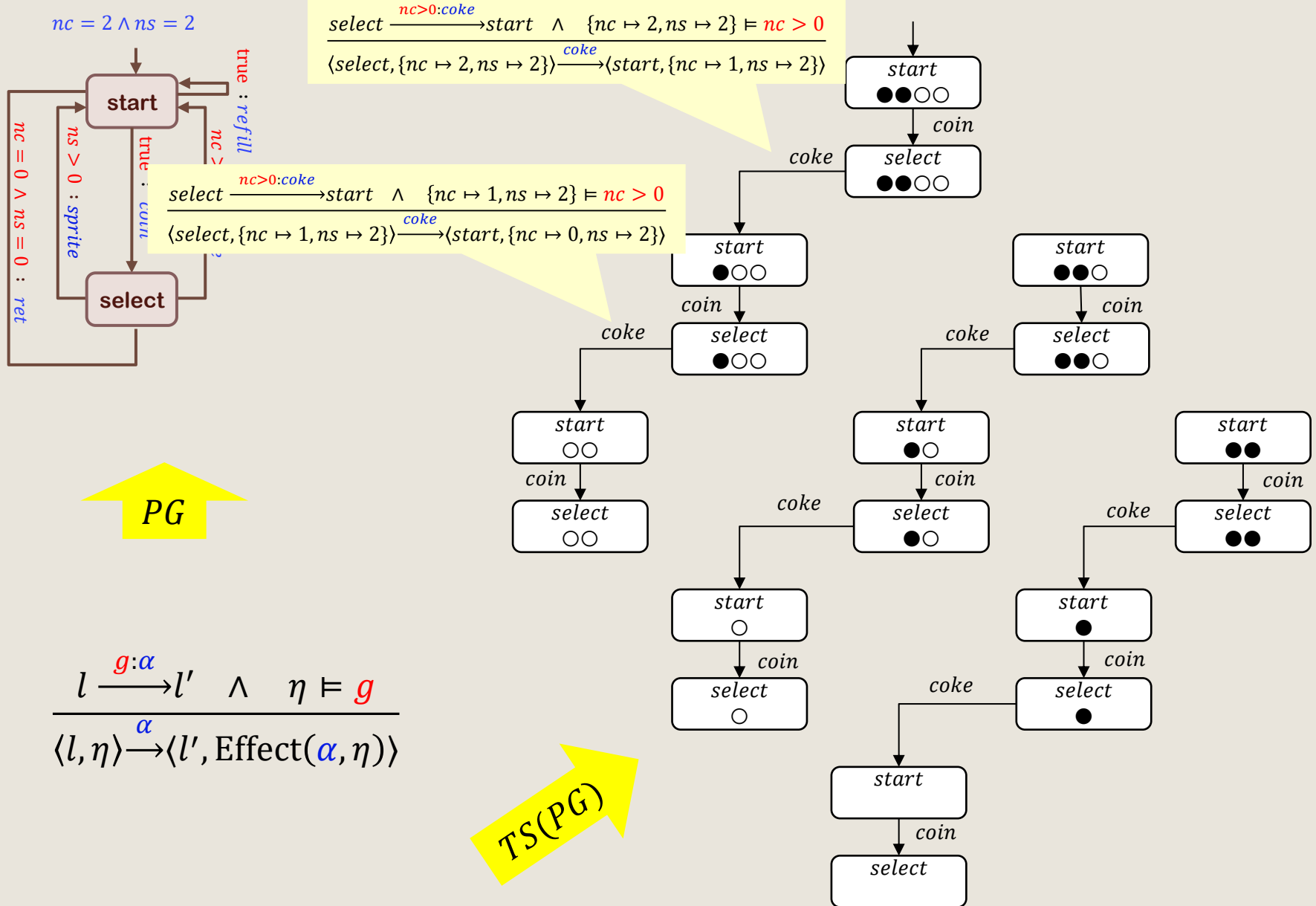


PG

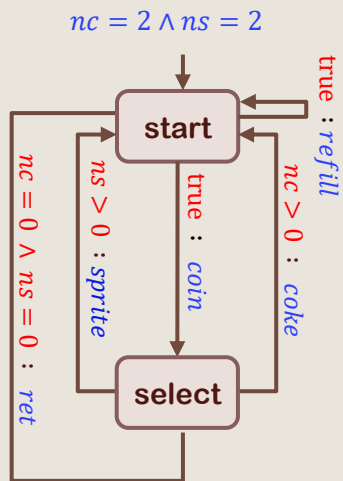
$$\frac{l \xrightarrow{g:\alpha} l' \quad \wedge \quad \eta \models g}{\langle l, \eta \rangle \xrightarrow{\alpha} \langle l', \text{Effect}(\alpha, \eta) \rangle}$$

TS(PG)

דוגמה: פריסת מערכת המעברים למכונת המשקאות



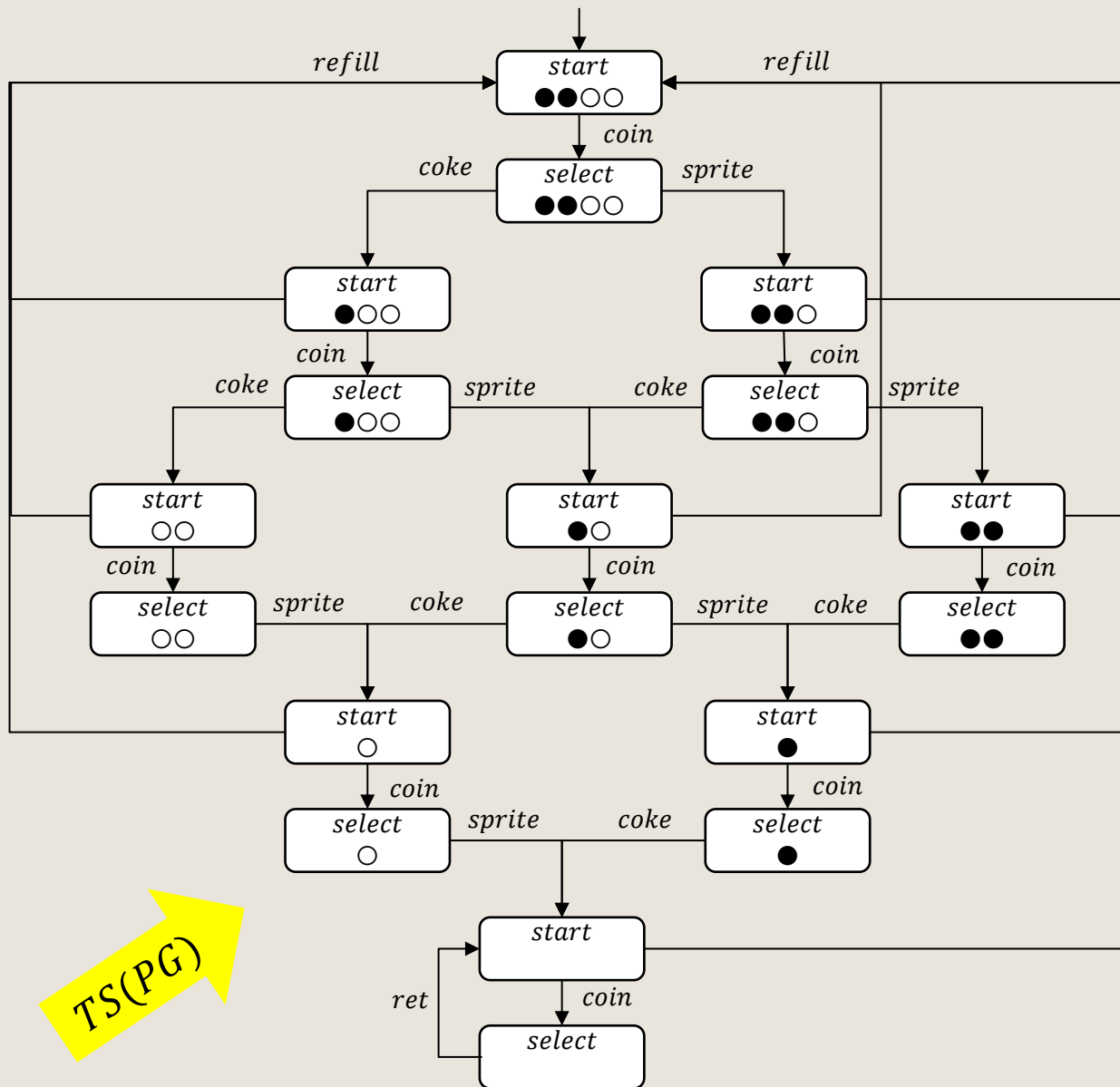
דוגמה: פריסת מערכת המעברים למכונת המשקאות



PG

$$\frac{l \xrightarrow{g:\alpha} l' \quad \wedge \quad \eta \models g}{\langle l, \eta \rangle \xrightarrow{\alpha} \langle l', \text{Effect}(\alpha, \eta) \rangle}$$

TS(PG)





חישוב אוטומטי של הגרף שהוצג בשקף הקודם

בניית גרף התוכנית ⇐

```
ProgramGraph pg = ExtendedVendingMachineBuilder.build();
```

פונקציית פירוש פעולות ⇐

```
Set<ActionDef> actDef =  
    ExtendedVendingMachineBuilder.getActionDefs();
```

פונקציית פירוש תנאים ⇐

```
Set<ConditionDef> condDef =  
    ExtendedVendingMachineBuilder.getConditionDefs();
```

פריסת גרף התוכנית למערכת מעברים ⇐

```
TransitionSystem ts =  
    transitionSystemFromProgramGraph(pg, actDef, condDef);
```

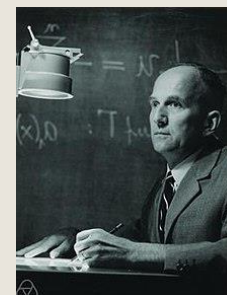
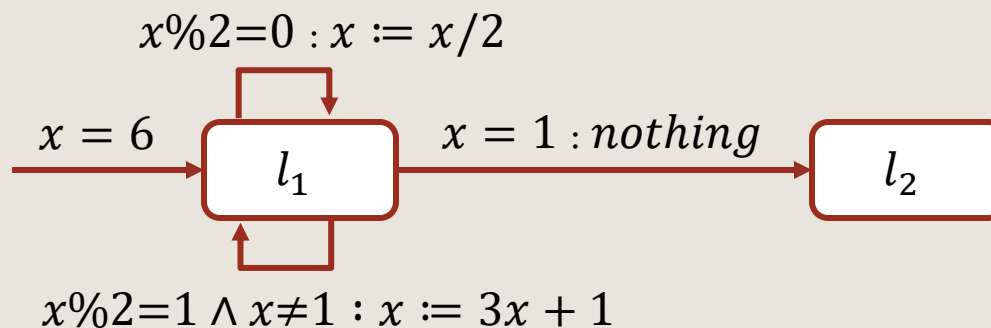


השיטה שאתם אמורים לממש

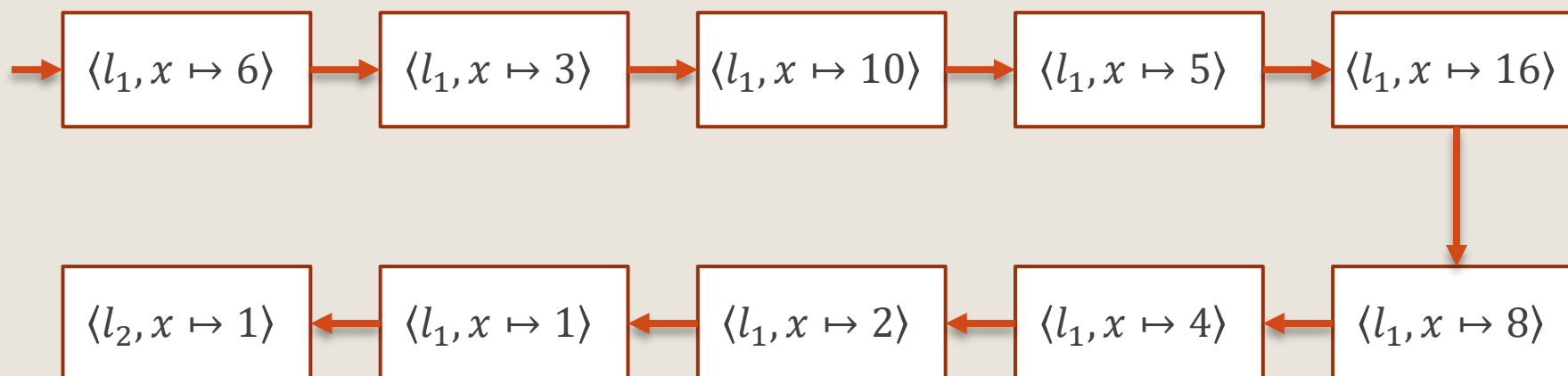


דוגמה: השערת קולץ (Collatz)

PG:



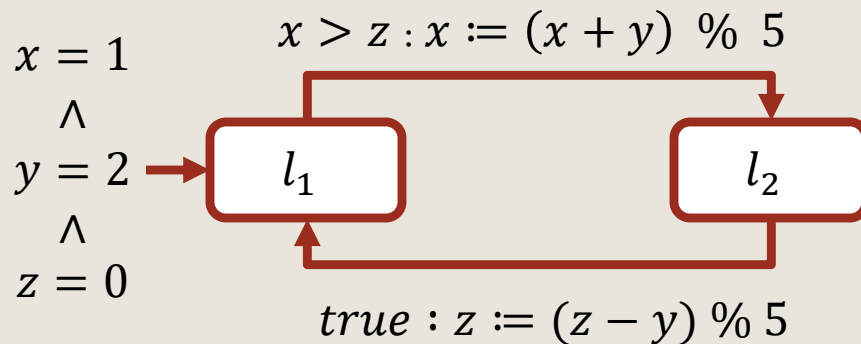
TS(PG):



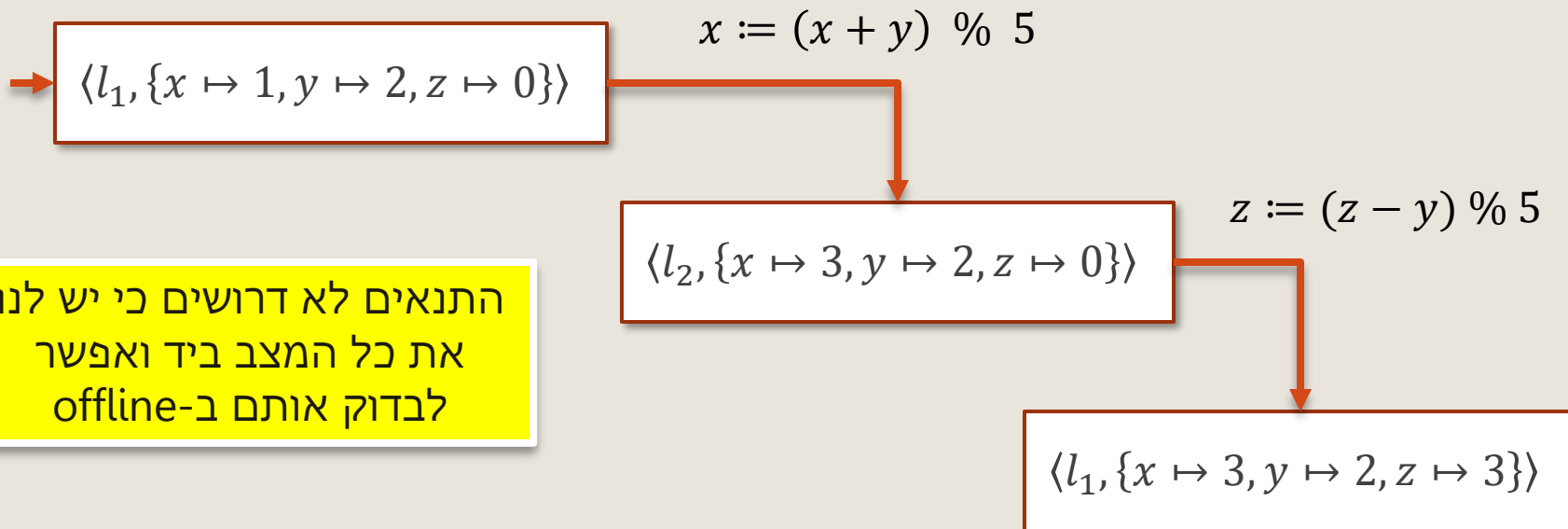


דוגמה: במערכת המעברים אין תנאים

PG:



TS(PG):

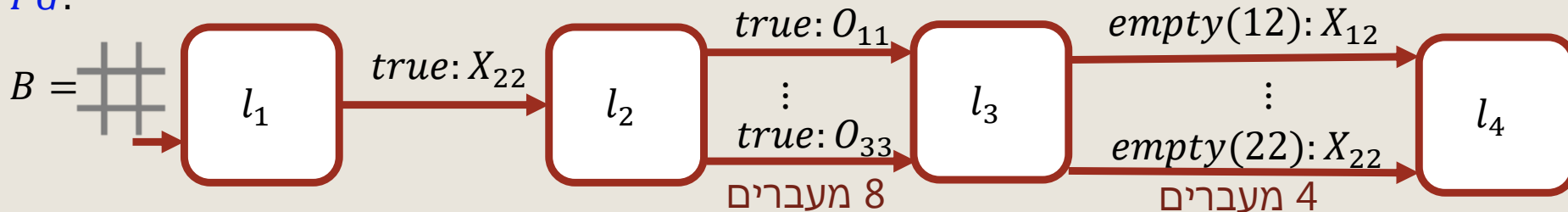


התנאים לא דרושים כי יש לנו
 את כל המצב ביד ואפשר
 לבדוק אותם ב-offline

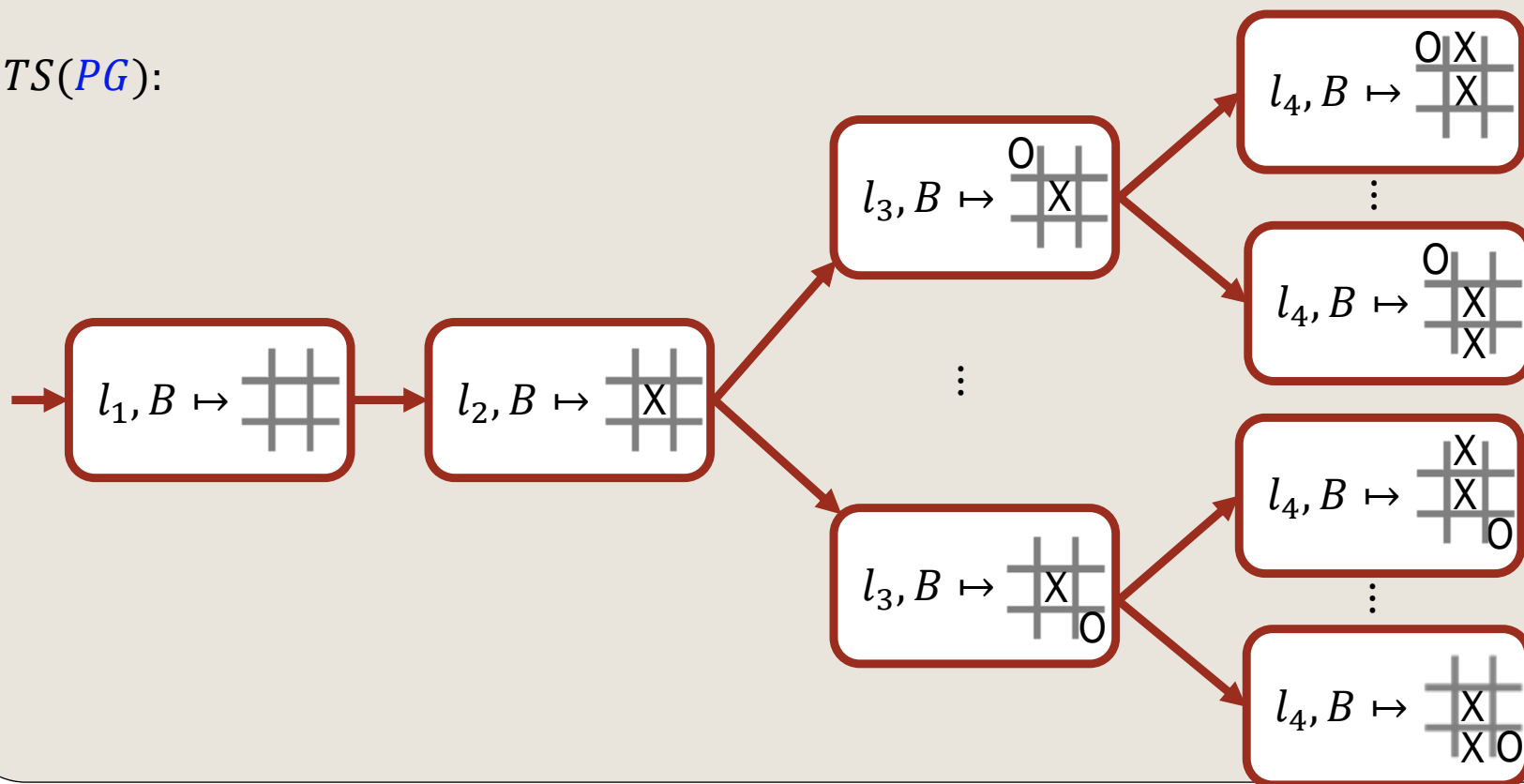


דוגמה: משחק איקס-עיגול

PG:



$TS(PG)$:





סיכום

- גרף תוכנית הוא מודל המתאר את הקוד של התוכנית
- נתרגם גרף תוכנית PG למערכת מעברים $TS(PG)$ שאותה נבדוק
- מצב של מערכת המעברים $TS(PG)$ = מקום בתוכנית + ערכי המשתנים
- מעברים ב- $TS(PG)$ מתאימים לביצוע פקודה בתוכנית. ייתכן שישתנו ערכי המשתנים ו/או המקום הבא לביצוע
- ב- $TS(PG)$ **אין תנאים** כי אנחנו מקודדים באופן מפורש את כל המעברים האפשריים. אם התנאי בתוכנית לא מתקיים עבור ההשמה המסוימת שמתאר המצב, לא נמתח חץ.
- השלב הבא: **תוכניות עם ריבוי תהליכים (multi threaded)**